Napisy w CPP

1)Wprowadzenie

W języku C++ zdefiniowana jest **klasa string**, dostępna po dołączeniu pliku **nagłówkowego string**.

#include <string>

Pozwala ona na tworzenie napisów i manipulowanie.

2)Konstruktory

Obiekt klasy string można utworzyć na kilka sposobów:

string( )
string(const string& wzor, size_type start = 0, size_type ile = npos)
string(const char* wzor)
string(const char* wzor, size_type ile)
string(size_type ile, char c)
string(const char* start, const char* kon)

są przeciążonymi konstruktorami klasy string. W ostatnim z nich typem argumentów może być dowolny iterator wskazujący na znaki: w najprostszym przypadku są to po prostu wskaźniki do znaków C-napisu.

Przykłady

string s;	tworzy napis pusty;
string s1 = "Acapulco"; string s2("Acapulco");	tworzy napis zainicjowany kopią C-napisu;
string s("Acapulco",n);	tworzy napis zainicjowany kopią pierwszych n znaków C-napisu podanego jako pierwszy argument. Argument n jest typu size_type;
string s(n,'x');	tworzy napis zainicjowany n powtórzeniami znaku (w tym przypadku znaku 'x'). Argument n jest typu size_type. Dla n= npos zgłasza wyjątek length_error. Zauważmy, że nie ma konstruktora pobierającego pojedynczy znak jako jedyny argument. Nie jest to wielka strata, gdyż zawsze możemy użyć powyższego konstruktora w formie s(1,'x').
Jeśli s jest obiektem klasy string, to	
string s1(s);	tworzy napis zainicjowany kopią napisu s (jest to wywołanie konstruktora kopiującego);
string s2(s,n);	tworzy napis zainicjowany kopią napisu s poczynając od znaku na pozycji n, licząc pozycje, jak zwykle, od zera. Argument n jest typu size_type. Jeśli ma wartość większą lub równą długości napisu s, to zgłaszany jest wyjątek out_of_range. Zauważmy różnicę:

	jeśli s byłoby C-napisem, to n miałoby interpretację liczby znaków licząc od początku!
string s2(s,n,k);	tworzy napis zainicjowany kopią napisu s poczynając od pozycji n i uwzględniającej co najwyżej k znaków. Jeśli wartość n+k jest większa od długości napisu s to błędu nie ma: interpretowane to jest jako wszystkie znaki od pozycji n. W szczególności wartością k może być npos. Wszystkie trzy ostatnie przypadki są implementowane w postaci jednego konstruktora o dwóch parametrach domyślnych:

```
string(const string& s, size_type start = 0,
                        size_type   ile = npos);
```

Jest też konstruktor wykorzystujący jawnie fakt, że obiekt klasy string jest kolekcją znaków. Jego szczególnym przypadkiem jest konstruktor, którego dwoma argumentami są dwa wskaźniki do znaków w zwykłym C-napisie: nowo utworzony napis C++ zainicjowany zostanie ciągiem znaków od tego wskazywanego przez pierwszy argument (włącznie) do znaku wskazywanego przez drugi argument, ale bez niego (czyli wyłącznie). Na przykład

```
const char* cnapis = "0123456789";
string s(cnapis+1,cnapis+7);
cout << s << endl;
```

wydrukuje '123456'. Jest to przykład bardziej ogólnego mechanizmu iteratorów, o których powiemy więcej w rozdziale o iteratorach . Na razie możemy iteratory traktować jako pewne uogólnienie wskaźników. Na przykład

```
string  s("Warszawa");
string  s1(s.begin()+3, s.end()-2);
cout << s1 << endl;
```

wydrukuje ' sza'. Metody begin i end zwracają iteratory wskazujące na pierwszy oraz na pierwszy za ostatnim znak napisu. Dodawanie do i odejmowanie od iteratorów liczb całkowitych działa podobnie jak dla wskaźników. Jak zwykle, pierwszy iterator wskazuje na pierwszy znak który ma być uwzględniony, podczas gdy drugi na pierwszy znak który ma już być opuszczony.

3)Metody i operatory

a)operatora przypisania.

```
const char* cstr = "strin";
string s1, s2, s3, s(" C++");
s1 = cstr;
s2 = 'g';
s3 = s;
cout << s1 << s2 << s3 << endl;
```

wydrukuje ' string C++'. Przypisanie jest głębokie, co znaczy, że na przykład po przypisaniu s1=s2 obiekt s1 jest całkowicie niezależny od obiektu s2: późniejsze zmiany s2 nie wpływają na s1 i vice versa.

b)konkatenowanie - operatora dodawania.

```
string s1 = "C";
const char* cn = "string";

string s = s1 + '-' + cn;
cout << s << endl;
```

utworzy i wypisze ' C-string', bo wynikiem pierwszego złożenia będzie napis C++ zawierający ' C-', który następnie zostanie złożony z C-napisem ' string'. Natomiast konstrukcja

```
const char* cn = "C";
string s1 = "string";

string s = cn + '-' + s1;
cout << s << endl;
```

byłaby błędna, gdyż pierwsze „dodawanie” dotyczyłoby C-napisu i znaku, a nie napisu C++.

Operator ' +=' dodaje prawy argument, który może być napisem C++, C-napisem lub pojedynczym znakiem, do napisu C++ będącego lewym argumentem.

c)Napis może też być traktowany jako tablica znaków, pierwszy znak ma indeks [0]

```
string s("Basia");
s[0] = 'K';
for (int i = 0; i < 5; i++) cout << s[i];
```

wyświetli napis ' Kasia'.

d)operatory porówniania ' ==', ' !=', ' >', ' >=', ' <', ' <=.

Przykład: Sortowanie napisów

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

void insertionSort(string[],int);

 int main(void) {
 int i;
 string krolowie[] = {
 string("Zygmunt"),   string("Michal"),
 string("Wladyslaw"), string("Anna"),
```

```
 string("Jan"),       string("Boleslaw")
                    };

    const int ile = sizeof(krolowie)/sizeof(string);

    insertionSort(krolowie, ile);

        for ( i = 0; i < ile; i++ )
            cout << setw(10) << krolowie[i] << endl;
     }

    void insertionSort(string a[], int wymiar) {
       if ( wymiar <= 1 ) return;

        for ( int i = 1 ; i < wymiar ; ++i ) {
             int j = i;
             string v = a[i];
            while ( j >= 1 && v < a[j-1] ) {
                a[j] = a[j-1];
                 j--;
            }
            a[j] = v;
        }
      }
```

e)operatory wstawiania i wyjmowania ze strumienia, ' <<' i ' >>'.

operator '>>' działa tak, że pomijane są wiodące białe znaki, a wczytywanie kończy się po napotkaniu pierwszego białego znaku za napisem - nie da się więc wczytać w ten sposób napisu złożonego z wielu słów.

f)określanie długości napisu

size( )
length( )

Na przykład:
jeśli napis="ZSE", to napis.size() zwróci 3.

g)sprawdzenie czy napis jest pusty

bool empty( ) - zwraca, w postaci wartości logicznej, odpowiedź na pytanie czy napis jest pusty?

h)wyciagniecie określonego znaku z łańcucha znaków

char& **at(size_type n)** - zwraca odniesienie do n-tego znaku (licząc od zera) z napisu, na rzecz którego została wywołana. Zakres jest sprawdzany: jeśli n jest większe od lub równe długości napisu, wysyłany jest wyjątek out_of_range. Metoda ta zatem ma działanie podobne

do operatora indeksowania, ale, ze względu na sprawdzanie zakresu, jest mniej efektywna, choć bardziej bezpieczna.

Na przykład string("Ula").at(2) zwraca literę 'a'.

i)zmian długości napisu

void **resize(size_type n, char c = '\0')** - zmienia rozmiar napisu na n. Jeśli n jest mniejsze od aktualnej długości napisu, pozostałe znaki są usuwane. Jeśli n jest większe od długości napisu, napis jest uzupełniany do długości n znakami c - domyślnie znakami ' \0'.

j)kasowanie znaków z napisu

void **clear( )** - usuwa wszystkie znaki z napisu, pozostawiając go pustym. Równoważna wywołaniu metody resize(0).

k)wycinanie ciagu znaków z napisu

string **substr(size_type start = 0, size_type ile = npos)** - zwraca napis będący podciągiem napisu na rzecz którego metoda została wywołana. Podciąg składa się ze znaków od pozycji start i liczy ile znaków. Jeśli start+ile jest większe niż długość napisu, to błędu nie ma; do podciągu brane są wszystkie znaki napisu od tego na pozycji start.

Na przykład

```
string s("Pernambuco");
cout << s.substr(5,3) << endl;
```

wypisze ' mbu'.

l)kopiowanie ciągu znaków

size_type **copy(char cn[], size_type ile, size_type start = 0)** - kopiuje do C-napisu cn podciąg złożony z ile znaków, poczynając od tego na pozycji start. Zwraca liczbę przekopiowanych znaków. Znak ' \0' nie jest dostawiany. Zwróćmy uwagę na kolejność argumentów start i ile - odwrotną niż w metodzie substr.

Na przykład

```
char nap[] = "xxxxxx";
string s("Barbara");
string::size_type siz = s.copy(nap,3,2);
cout << "Skopiowano " << siz << " znaki: \n"
    << nap << endl;
```

wypisze ' Skopiowano 3 znaki: rbaxxx'.

m)zmiana ciagu znaków

void swap(string s1) - zamienia napis s1 z tym, na rzecz którego metodę wywołano. Na przykład

```
string s("Arles"), s1("Berlin");
s.swap(s1);
cout << s << " " << s1 << endl;
```

wypisze ' Berlin Arles'.

n)napis według wzorca

string& **assign(const string& wzor)**
string& assign(const char* wzor)
string& assign(string wzor, size_type start, size_type ile)
string& assign(const char* wzor, size_type ile)
string& assign(size_type ile, char c)

string& **assign(const char* start, const char* kon)** - ustala zawartość napisu i zwraca odniesienie do niego (zastępuje operator przypisania). Argumenty mają podobną postać jak dla konstruktorów. W ostatniej metodzie typem argumentów może być iterator wskazujący na znaki: na przykład są to po prostu wskaźniki do znaków C-napisu.
W najprostszym przypadku argumentem jest inny napis w postaci napisu C++ lub C-napisu; tak więc po

```
string s1("xxx"), s2("Zuzia");
s1.assign(s2);
s2.assign("Kasia");
```

wartością s1 będzie ' Zuzia' a zmiennej s2 ' Kasia'.
Tak jak dla konstruktorów, jako argument można podać podnapis napisu C++ o podanej pozycji początku i długości - za duża długość znaczy aż do końca. Można też podać podnapis C-napisu złożony z podanej liczby znaków licząc od początku:

```
string s1("0123456789"), s2;
const char* p = "0123456789";
s1.assign(s1,2,5);
s2.assign(p, 5);
cout << s1 << " " << s2 << endl;
```

wypisze ' 23456 01234'.
Za pomocą metody assign można utworzyć napis złożony z ile powtórzeń znaku (piąta forma z wymienionych powyżej). Można też użyć wskaźników do znaków w C-napisie jako iteratorów wyznaczających podciąg; trzeba tylko pamiętać, że wskazywany podnapis nie zawiera wtedy znaku wskazywanego jako górne ograniczenie podciągu:

```
string s1("0123456789"), s2;
const char* p = "0123456789";
s1.assign(5,'x');
s2.assign(p+3,p+5);
cout << s1 << " " << s2 << endl;
```

wypisze ' xxxxx 34'.

o)wstawianie tekstu

string& **insert(size_type gdzie, const string* wzor)**
string& insert(size_type gdzie, const char* wzor)
string& insert(size_type gdzie, string wzor, size_type start, size_type ile)
string& insert(size_type gdzie, const char* wzor, size_type ile)
string& insert(size_type gdzie, size_type ile, char c) - modyfikuje napis i zwraca odniesienie do niego, wstawiając na pozycji o indeksie gdzie znaki z innego napisu, opisywanego przez pozostałe argumenty. Znaki od pozycji gdzie są „przesuwane” w prawo za fragment wstawiony. Znaczenie argumentów określających ciąg znaków do wstawienia jest takie samo jak dla metody assign. Na przykład

```
string s1("mama"), s2("plastyka");
const char* p = "temat";
s1.insert(2,p,2).insert(6,s2,4,4);
cout << s1 << endl;
```

wypisze ' matematyka'.
Prócz tych form istnieją jeszcze trzy inne formy tej metody:

iterator insert(iterator gdzie, char c)
void insert(iterator gdzie, size_type ile, char c)
void insert(iterator gdzie, const char* start, const char* kon) - gdzie pierwszym argumentem jest iterator (uogólniony wskaźnik) do znaku w napisie, przed który wstawiany jest ciąg określany przez pozostałe argumenty. W trzeciej z tych form rolę dwóch ostatnich argumentów mogą pełnić nie tylko wskaźniki do znaków, ale ogólnie iteratory wskazujące na znaki (na przykład iteratory typu string::iterator). Na przykład

```
string s1("abbccdd");
s1.insert(s1.insert(s1.begin()+5,'c')+1,2,'d');
cout << s1 << endl;
```

wypisze ' abbcccdddd'.

p)modyfikacja napisu

string& append(const string& wzor)
string& append(const string& wzor, size_type start, size_type ile)
string& append(const char* wzor)
string& append(const char* wzor, size_type ile)
string& append(size_type ile, char c)
string& append(const char* start, const char* kon) - modyfikuje napis i zwraca odniesienie do niego, dodając na końcu tego napisu napis określany przez argumenty. Znaczenie argumentów określających ciąg znaków do wstawienia jest takie samo jak dla metod insert. W ostatniej metodzie typem argumentów może być dowolny iterator wskazujący na znaki: tu są to po prostu wskaźniki do znaków C-napisu.

r)usuwanie fragmentu tekstu

string& erase(size_type start = 0, size_type ile = npos)
iterator erase(iterator start)
iterator erase(iterator start, iterator kon) - usuwa fragment napisu, zwracając odniesienie do zmodyfikowanego napisu lub iterator odnoszący się do zmodyfikowanego napisu i wskazujący na pierwszy znak za fragmentem usuniętym. Pierwsza forma usuwa ile znaków (domyślnie npos, czyli wszystkie) od pozycji start (domyślnie od pozycji zerowej). Druga forma usuwa wszystkie znaki od pozycji wskazywanej przez iterator start, a trzecia od znaku wskazywanego przez iterator start do znaku poprzedzającego znak wskazywany przez iterator kon. Na przykład

```
string s("0123456789");
string::iterator it = s.erase(s.begin()+3,s.end()-3);
cout << s << " " << *it << endl;
```

wypisze ' 012789 7'.

s)zamiana tekstu

string& replace(size_type start, size_type ile, const string& wzor)
string& replace(size_type start, size_type ile, const string& wzor, size_type s, size_type i)
string& replace(size_type start, size_type ile, const char* wzor, size_type i)
string& replace(size_type start, size_type ile, const char* wzor)
string& replace(size_type start, size_type ile, size_type i, char c)
string& replace(iterator start, iterator kon, const string& wzor)
string& replace(iterator start, iterator kon, const char* wzor)
string& replace(iterator start, iterator kon, const char* wzor, size_type i)
string& replace(iterator start, iterator kon, size_type i, char c)
string& replace(iterator start, iterator kon, const char* st1, const char* kn1) - usuwa fragment napisu określony pierwszymi dwoma argumentami i wstawia na to miejsce napis określony pozostałymi argumentami. W ostatniej z tych metod typem dwóch ostatnich argumentów może być dowolny iterator wskazujący na znaki: w najprostszym przypadku są to po prostu wskaźniki do znaków C-napisu. Zasady określania napisów lub podnapisów są te same co dla metod insert. Metody replace zwracają odniesienie do zmodyfikowanego napisu. Na przykład

```
string s("0123456789");
const char* p("abcdef");
s.replace(0,2,p,2).replace(s.end()-2,s.end(),p+4,p+6);
cout << s << endl;
```

wypisze ' ab234567ef'.

r)znajdowanie tekstu

size_type find(const string* s, size_type start = 0)
size_type find(const char* p, size_type start = 0)
size_type find(const char* p, size_type start, size_type ile)
size_type find(char c, size_type start = 0)
size_type rfind(const string* s, size_type start = npos)

size_type rfind(const char* p, size_type start = npos)
size_type rfind(const char* p, size_type start, size_type ile)
size_type rfind(char c, size_type start = npos) - szukają, poczynając od pozycji start, podnapisu określonego przez pozostałe argumenty. Rodzina metod rfind działa analogicznie, ale przeglądanie odbywa się od pozycji startowej w kierunku początku napisu. Wszystkie metody zwracają pozycję (indeks) pierwszego znaku poszukiwanego podnapisu w napisie przeszukiwanym. Jeśli przeszukiwanie zakończyło się porażką, zwracane jest npos. W przykładzie poniżej find szuka w napisie ' abc345abcAB' poczynając od pozycji 3 (czyli od cyfry '3') napisu złożonego z dwóch pierwszych znaków C-napisu p (czyli napisu ' ab'):

```
string s("abc345abcAB");
const char* p("abcdef");
string::size_type i = s.find(p,3,2);
cout << s.substr(i-1,5) << endl;
```

Fragment wypisuje ' 5abcA'.

t)znajdowanie tekstu od pozycji

size_type find_first_of(/* args */)
size_type find_last_of(/* args */)
size_type find_first_not_of(/* args */)
size_type find_last_not_of(/* args */) - mają typ wartości i argumentów takie same jak odpowiednie metody find i rfind. Pierwsze dwie metody szukają, poczynając od pozycji start, pierwszego wystąpienia jakiegokolwiek znaku należącego do napisu określonego przez pozostałe argumenty. Kierunek przeszukiwania dla metod z _last_ w nazwie jest od pozycji start wstecz, a dla metod z _first_ w nazwie - do przodu. Metody z drugiej pary, te z _not_ w nazwie, działają podobnie, ale szukają wystąpienia znaku nie należącego do napisu określonego przez pozostałe argumenty. Jeśli odpowiedni znak został znaleziony, zwracana jest jego pozycja; jeśli nie, zwracane jest npos. Na przykład

```
1. string s("abc123.,!");
2. const char* p = "!.,?:1234";
3. string::size_type i = s.find_first_of(p);
4. string::size_type k = s.find_last_not_of(p,s.size()-1,5);
5. string s1(s.begin()+i,s.begin()+k+1);
6. cout << i << " " << k << " " << s1 << endl;
```

wypisze ' 3 5 123'. W linii czwartej jako drugi argument podaliśmy s.size()-1, bo przeszukiwanie odbywa się do tyłu, a więc zacząć trzeba od znaku ostatniego, a nie od pierwszego. Uwzględniamy przy tym tylko 5 pierwszych znaków ze wzorca p, a więc znaki ' !.,?:'. W linii piątej tworzymy nowy obiekt klasy string i inicjujemy go wycinkiem napisu s. W drugim argumencie dodajemy do s.begin()+k jedynkę, gdyż wycinek zawiera znaki tylko do poprzedzającego ten wskazywany przez drugi iterator.

u)porównywanie tekstów

int compare(const string& wzor)
int compare(size_type start, size_type ile, const string& wzor)
int compare(size_type start, size_type ile, const string& wzor,

size_type s, size_type ile1)
int compare(const char* p)
int compare(size_type start, size_type ile, const char* p, size_type i = npos) - porównują napis lub jego podciąg określony przez start i ile z napisem wyznaczonym przez pozostałe argumenty. Wynikiem jest -1, jeśli napis porównywany jest leksykograficznie wcześniejszy od napisu podanego jako argument, zero, jeśli są identyczne, a +1, jeśli jest leksykograficznie późniejszy.

w)wstawianie na koniec tekstu

void **push_back(char c)** - wstawia na koniec napisu znak c - s.push_back(c) działa jak wywołanie s.insert(s.end(),c), tyle że jest bezrezultatowe (metoda insert zwraca przy takim wywołaniu iterator). Ten sam efekt można uzyskać za pomocą metody append lub operatora ' +='.

z)inne metody

const char* **c_str( )** - zwraca wskaźnik do stałego C-napisu złożonego ze znaków napisu C++, na rzecz którego była wywołana. C-napis kończy się znakiem ' \0', a zatem może być argumentem funkcji operujących na zwykłych C-napisach. Pamiętać tylko należy, że zwracany wskaźnik jest typu const, a więc w razie konieczności modyfikacji uzyskany C-napis trzeba przekopiować do zwykłej, modyfikowalnej tablicy znaków.

iterator begin( ) - zwraca iterator (uogólniony wskaźnik, patrz rozdział o iteratorach) wskazujący na pierwszy znak napisu.

iterator end( ) - zwraca iterator wskazujący na znak pierwszy za ostatnim napisu.

reverse_iterator rbegin( )
reverse_iterator rend( ) - zwraca iterator „odwrotny" wskazujący na początek i koniec napisu w odwrotnym porządku. Na przykład po

```
string s1("korab");
string s2(s1.rbegin(),s1.rend());
```

s2 będzie zawierać napis ' barok'.

Prócz metod klasy string biblioteka włączana za pomocą pliku nagłówkowego string dostarcza bardzo przydatną funkcję (a więc nie metodę klasy):

istream& getline(istream& str, string& s)
istream& getline(istream& str, string& s, char eol) - wczytuje ze strumienia str jedną linię tekstu i wstawia ją do napisu s. Linia może zawierać białe znaki (prócz znaku końca linii). Znak końca linii jest wyjmowany ze strumienia, ale nie jest włączany do wynikowego napisu. Znak, który ma pełnić rolę znaku końca linii, można podać jako trzeci argument funkcji - domyślnie jest nim ' \n'. Funkcja zwraca odniesienie do strumienia str, tak więc nieco dziwna konstrukcja

```
string s1,s2;
getline(cin,s1) >> s2;
```